

Leveraging compilers for static software profiling.

Introduction to custom LLVM compiler passes for static code analysis.

CS6501/ECE6502 - Spring 2024

Saket Upadhyay,
PhD Student, Computer Science
SEAS, University of Virginia

The goal of this talk.

- **Introduce a new concept.**
- **Provide a codebase to play with.**

Program Compilation Brief

Understanding development
pipeline

```
#define AES_TE4_3(x) AES_PRECOMP_TE4_3[(x)]

#define CryptoUtils_POOL_SIZE (0x1 << 17) // 2^17

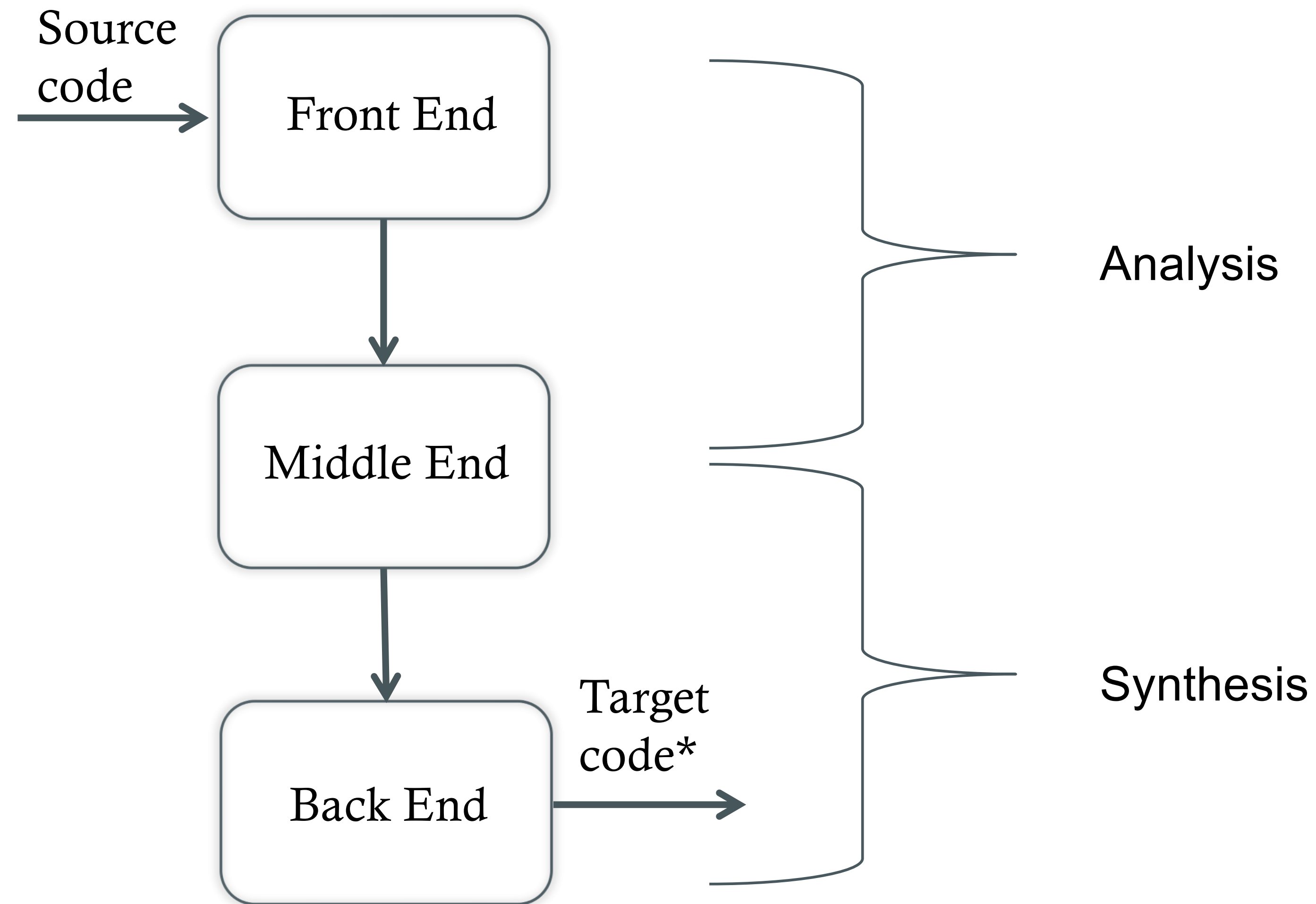
// SHA256
/* Various logical functions */
#define Ch(x, y, z) (z ^ (x & (y ^ z)))
#define Maj(x, y, z) (((x | y) & z) | (x & y))
#define __S(x, n) RORc((x), (n))
#define R1(x, n) (((x) & 0xFFFFFFFFUL) >> (n))
#define Sigma0(x) (__S(x, 2) ^ __S(x, 13) ^ __S(x, 22))
#define Sigma1(x) (__S(x, 6) ^ __S(x, 11) ^ __S(x, 25))
#define Gamma0(x) (__S(x, 7) ^ __S(x, 18) ^ R1(x, 3))
#define Gamma1(x) (__S(x, 17) ^ __S(x, 19) ^ R1(x, 10))
#define MIN(x, y) (((x) < (y)) ? (x) : (y))

#define RND(a, b, c, d, e, f, g, h, i, ki) \
    t0 = h + Sigma1(e) + Ch(e, f, g) + ki + W[i]; \
    t1 = Sigma0(a) + Maj(a, b, c); \
    d += t0; \
    h = t0 + t1;

#define RORc(x, y) \
    (((((unsigned long) (x) & 0xFFFFFFFFUL) >> (unsigned long) ((y) & 31)) \
    ((unsigned long) (x) << (unsigned long) (32 - ((y) & 31)))) & \
    0xFFFFFFFFUL)

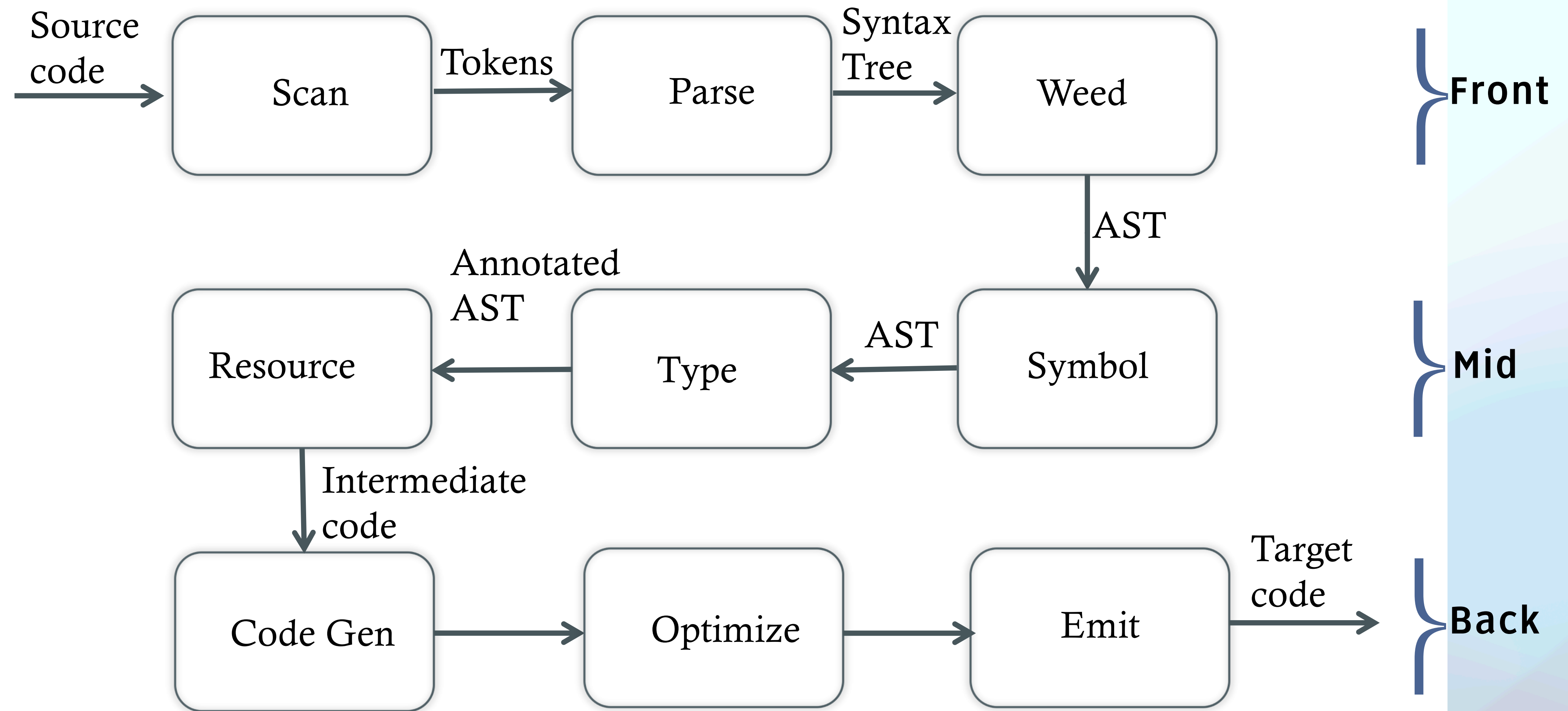
class CryptoUtils {
public:
```

Compiler Architecture

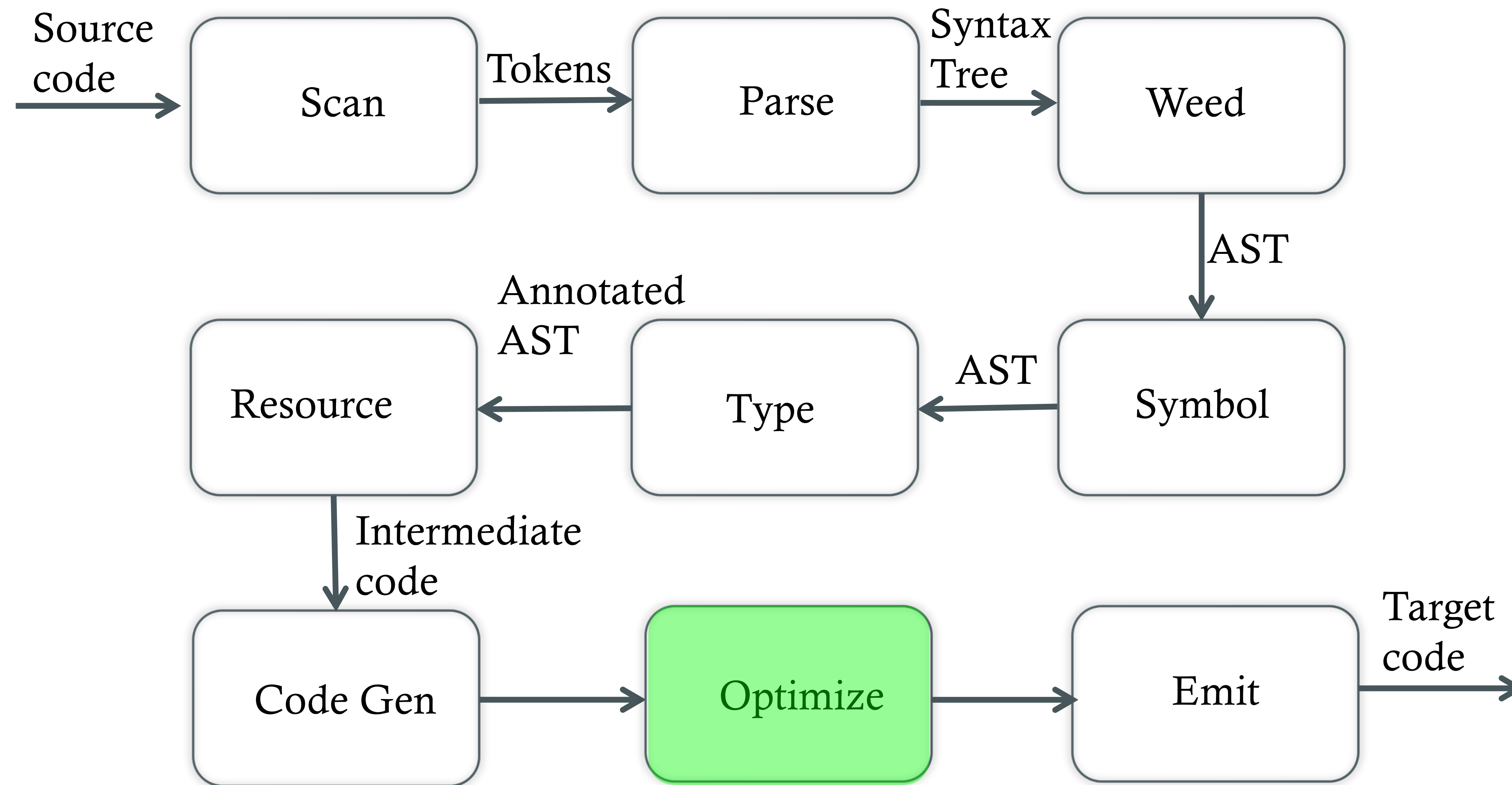


* Behaviorally equivalent code

Compiler Architecture



Our target : Optimization Phase

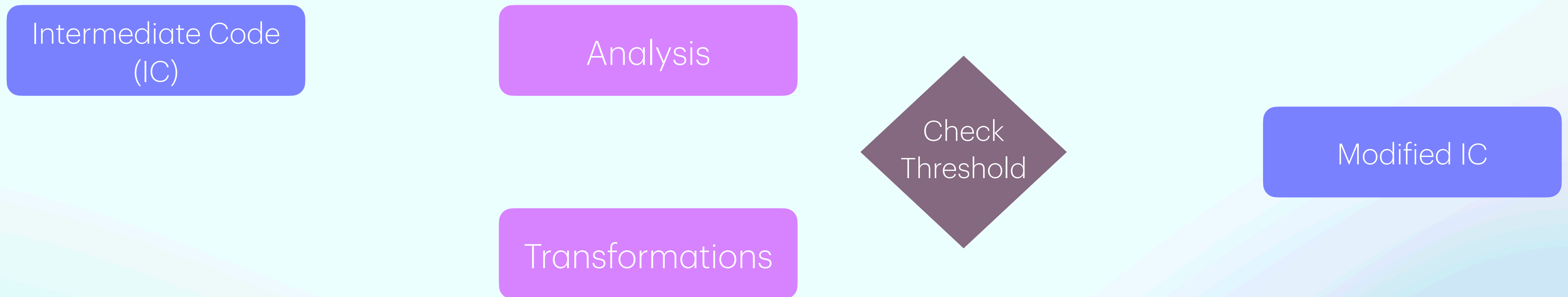


Optimization Phase

Analysis and Transformation

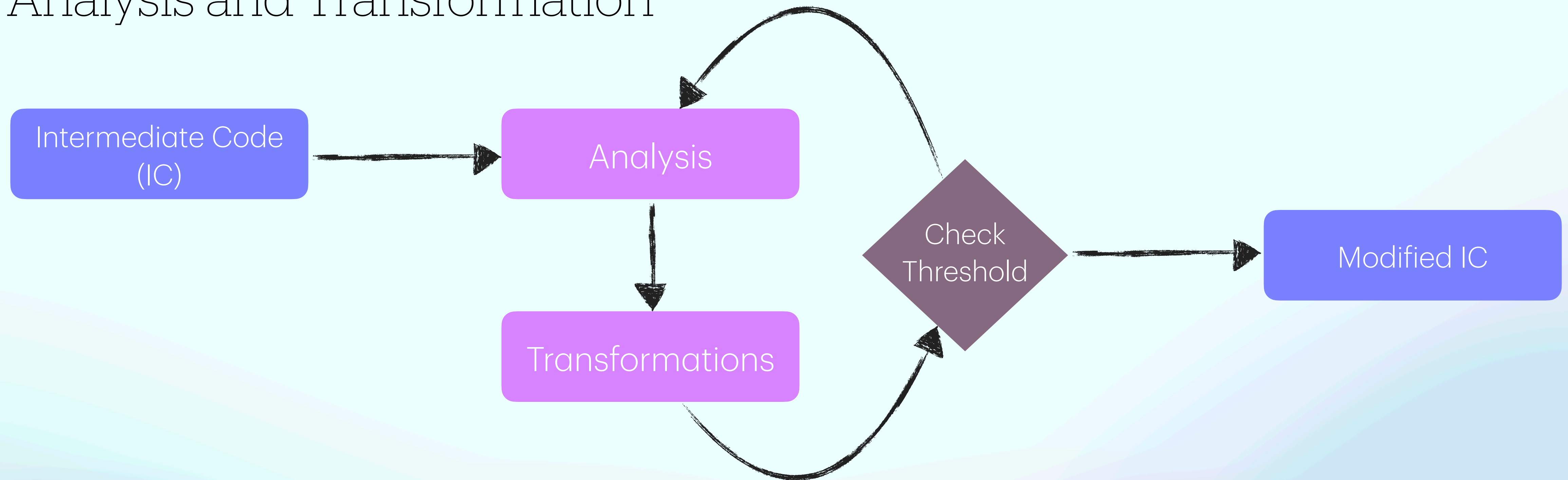
Optimization Phase

Analysis and Transformation



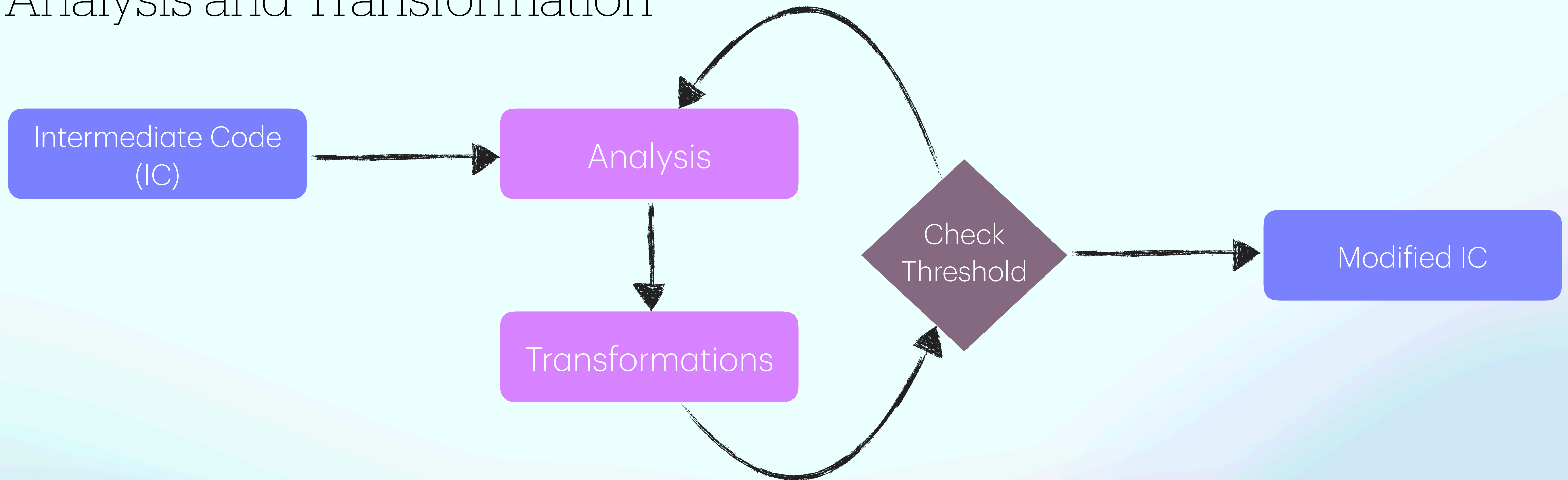
Optimization Phase

Analysis and Transformation



Optimization Phase

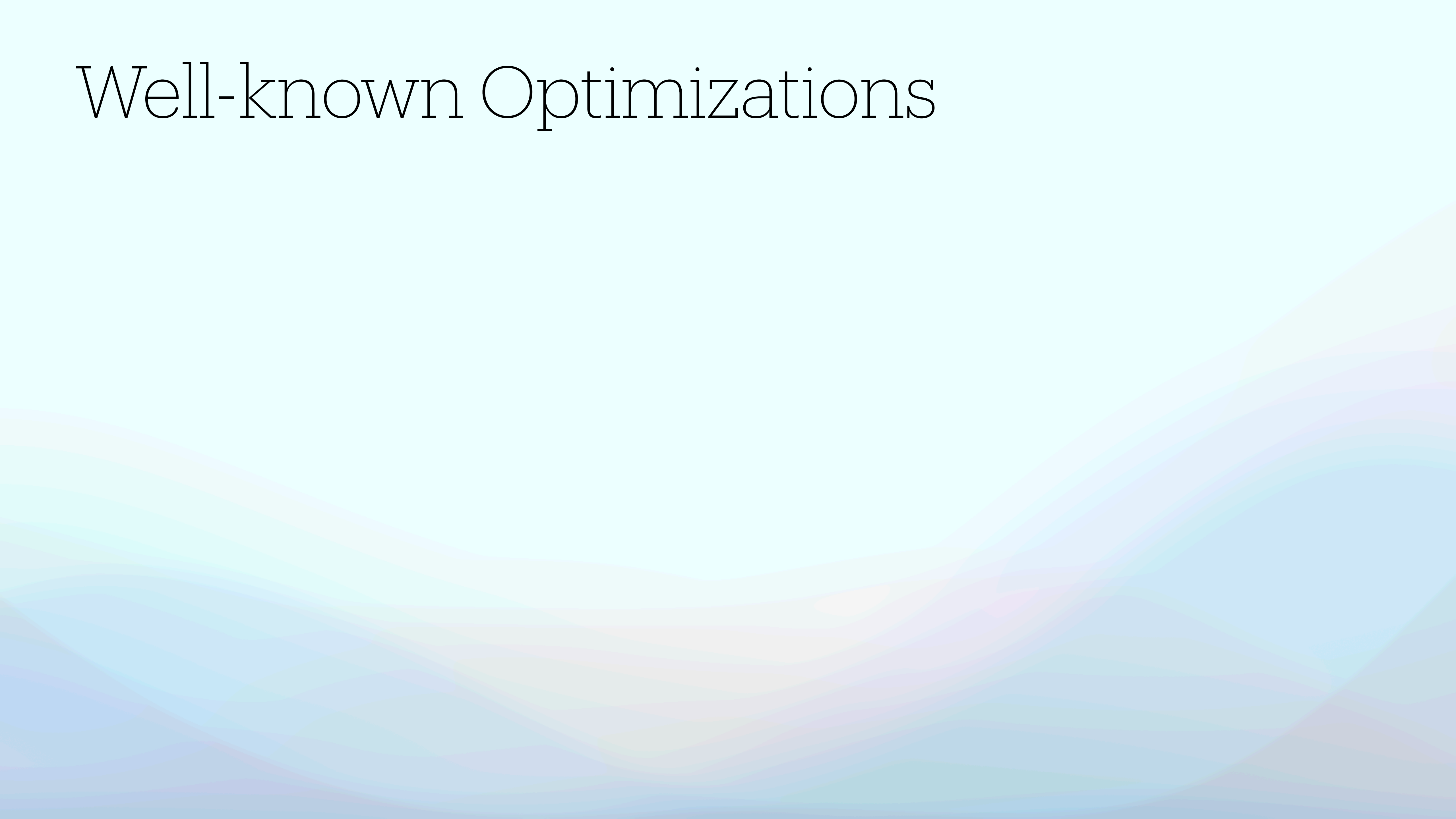
Analysis and Transformation



Analysis: Find optimization opportunities, tag optimization sites, queue passes, etc.

Transformation: Modify code, inject optimized logic, replace redundant code, etc.

Well-known Optimizations



Well-known Optimizations

- **Dead Code Elimination** :
 - Removes unused code blocks.

Well-known Optimizations

- **Dead Code Elimination** :

- Removes unused code blocks.

- **Loop Unrolling**:

- Flattens the loop, and removes unnecessary iterations.

Well-known Optimizations

- **Dead Code Elimination** :
 - Removes unused code blocks.
- **Loop Unrolling**:
 - Flattens the loop, and removes unnecessary iterations.
- **Function Inlining**:
 - Replace the function call with the contents of the function.

Well-known Optimizations

- **Dead Code Elimination** :
 - Removes unused code blocks.
- **Loop Unrolling**:
 - Flattens the loop, and removes unnecessary iterations.
- **Function Inlining**:
 - Replace the function call with the contents of the function.
- **Constant Folding**:
 - Computes expressions at compile time and replaces them with the result.

Well-known Optimizations

- **Dead Code Elimination** :
 - Removes unused code blocks.
- **Loop Unrolling**:
 - Flattens the loop, and removes unnecessary iterations.
- **Function Inlining**:
 - Replace the function call with the contents of the function.
- **Constant Folding**:
 - Computes expressions at compile time and replaces them with the result.
- **Instruction Scheduling**:
 - Reorders instructions to minimize pipeline stalls and maximize instruction-level parallelism.

LLVM

Infrastructure

A précis of LLVM.

```
[15]: 0x00010000UL, [16]: 0x00008000UL, [17]: 0x00004000UL, [18]: 0x00002000UL,
[20]: 0x00000800UL, [21]: 0x00000400UL, [22]: 0x00000200UL, [23]: 0x00000100UL,
[25]: 0x00000040UL, [26]: 0x00000020UL, [27]: 0x00000010UL, [28]: 0x00000008UL,
[30]: 0x00000002UL, [31]: 0x00000001UL};

CryptoUtils::CryptoUtils() { seeded = false; }

unsigned CryptoUtils::scramble32(const unsigned in, const char key[16]) {
    assert(key != NULL && "CryptoUtils::scramble key=NULL");

    unsigned tmpA, tmpB;

    // Orr, Nathan or Adi can probably break it, but who cares?

    // Round 1
    tmpA = 0x0;
    tmpA ^= AES_PRECOMP_TE0[((in >> 24) ^ key[0]) & 0xFF];
    tmpA ^= AES_PRECOMP_TE1[((in >> 16) ^ key[1]) & 0xFF];
    tmpA ^= AES_PRECOMP_TE2[((in >> 8) ^ key[2]) & 0xFF];
    tmpA ^= AES_PRECOMP_TE3[((in >> 0) ^ key[3]) & 0xFF];

    // Round 2
    tmpB = 0x0;
    tmpB ^= AES_PRECOMP_TE0[((tmpA >> 24) ^ key[4]) & 0xFF];
    tmpB ^= AES_PRECOMP_TE1[((tmpA >> 16) ^ key[5]) & 0xFF];
    tmpB ^= AES_PRECOMP_TE2[((tmpA >> 8) ^ key[6]) & 0xFF];
    tmpB ^= AES_PRECOMP_TE3[((tmpA >> 0) ^ key[7]) & 0xFF];

    // Round 3
    tmpA = 0x0;
    tmpA ^= AES_PRECOMP_TE0[((tmpB >> 24) ^ key[8]) & 0xFF];
    tmpA ^= AES_PRECOMP_TE1[((tmpB >> 16) ^ key[9]) & 0xFF];
    tmpA ^= AES_PRECOMP_TE2[((tmpB >> 8) ^ key[10]) & 0xFF];
    tmpA ^= AES_PRECOMP_TE3[((tmpB >> 0) ^ key[11]) & 0xFF];

    // Round 4
```

What is LLVM?



- **The LLVM Project is a collection of modular and reusable compiler and toolchain technologies.**
- **It can be used to create a frontend for a programming language or a backend for a target instruction set architecture.**

LLVM IR and Bitcode

Language independence

- **LLVM compiles any supported language into LLVM Intermediate Representation (IR).**
- **LLVM Bitcode (BC) refers to LLVM IR in its serialized binary format. LLVM can serialize IR to BC, which is a compact, binary representation of the program.**
- **LLVM IR serves as a common representation for programs before they're translated into machine code specific to a particular CPU architecture. It's designed to be simple, platform-independent, and efficient to manipulate.**



$M \rightarrow F \rightarrow BB \rightarrow I$

Building blocks of LLVM IR

$M \rightarrow F \rightarrow BB \rightarrow I$

Building blocks of LLVM IR

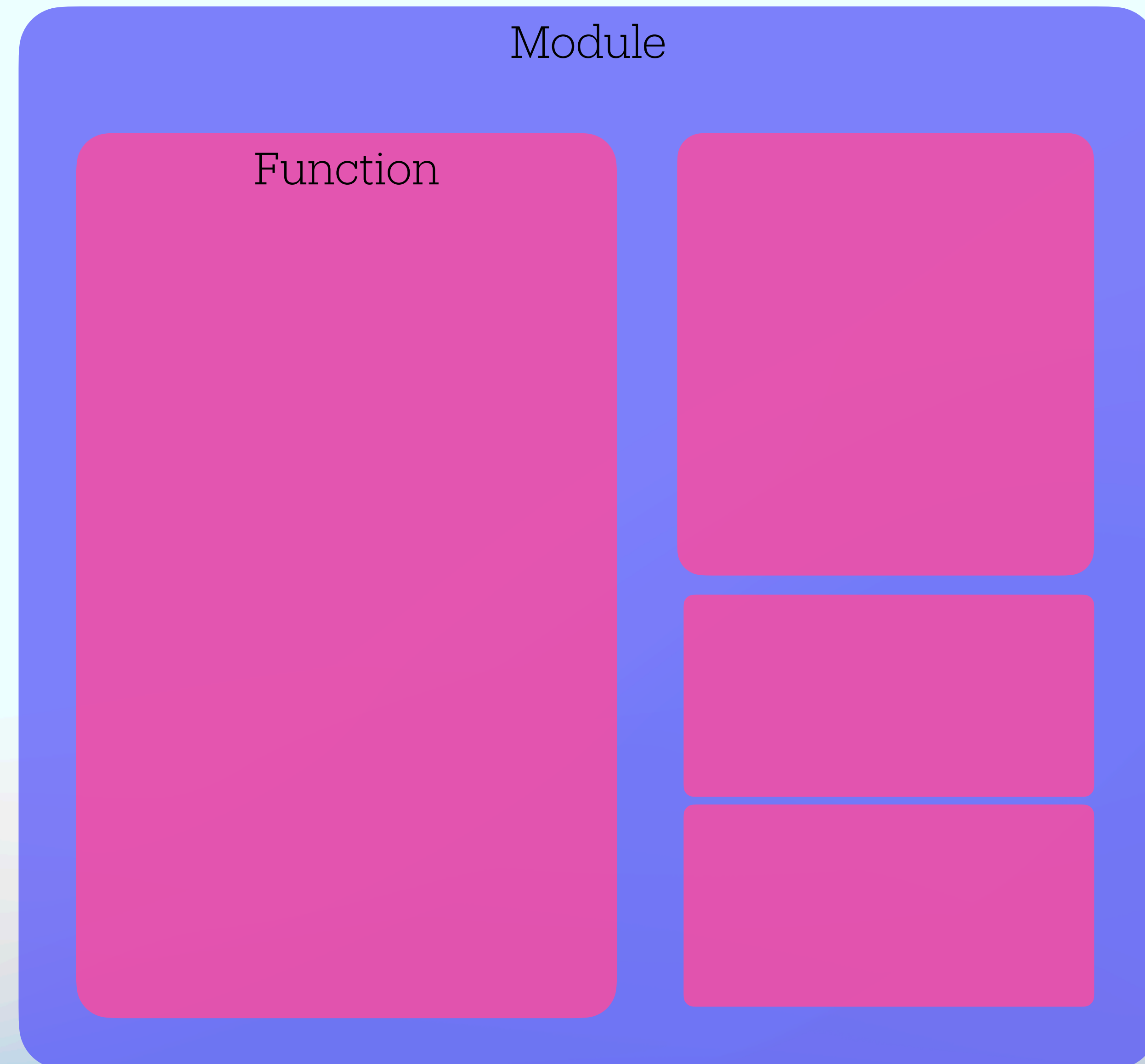
- **Modules** are the top-level container of all LLVM (IR) objects. Contains a list of global variables, list of functions, symbol table, etc.

Module

$$M \rightarrow F \rightarrow BB \rightarrow I$$

Building blocks of LLVM IR

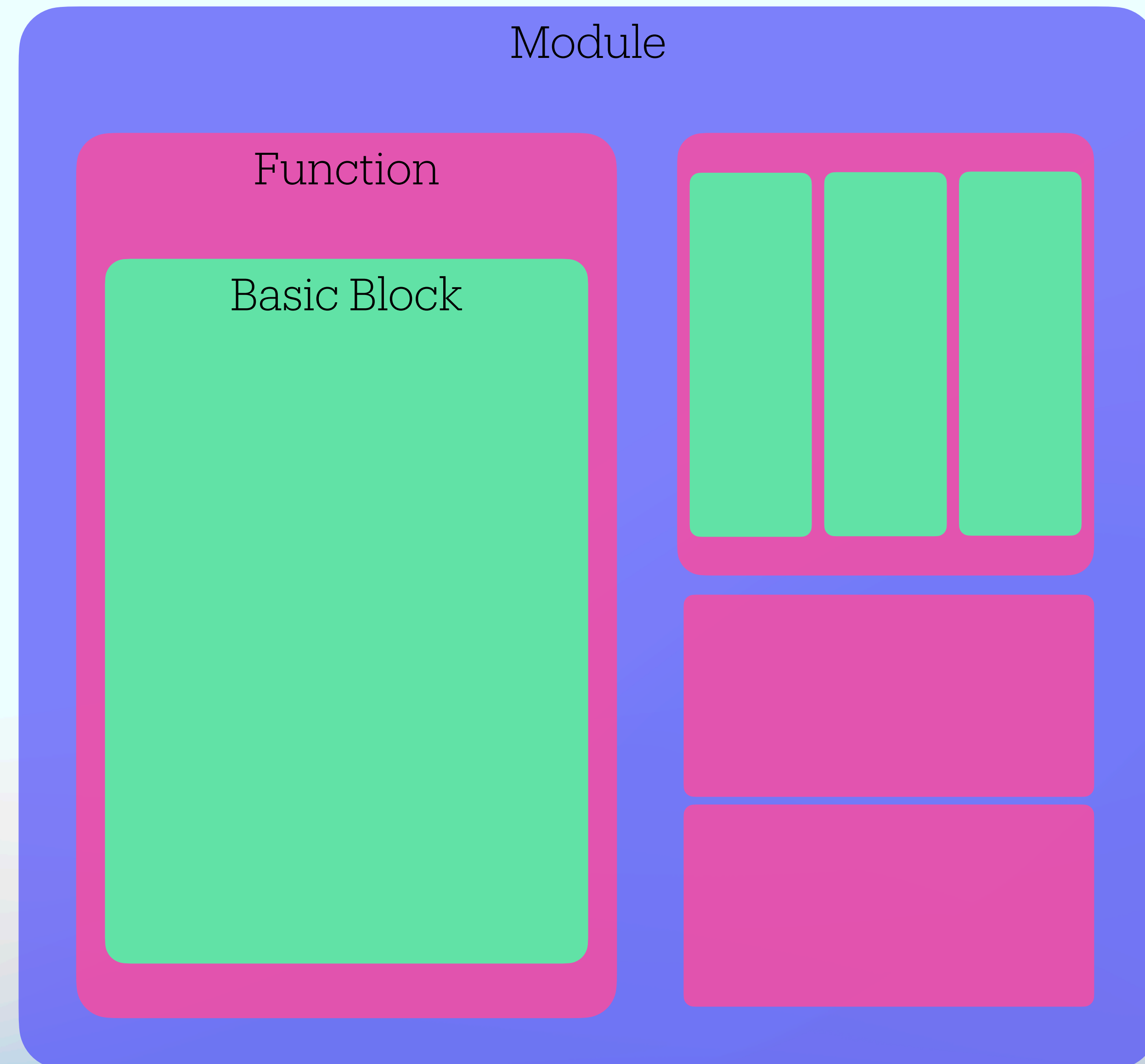
- **Modules** are the top-level container of all LLVM (IR) objects. Contains a list of global variables, list of functions, symbol table, etc.
- A **Function** is a procedure consisting of multiple basic blocks, usually connected via logical branches and jumps.



$M \rightarrow F \rightarrow BB \rightarrow I$

Building blocks of LLVM IR

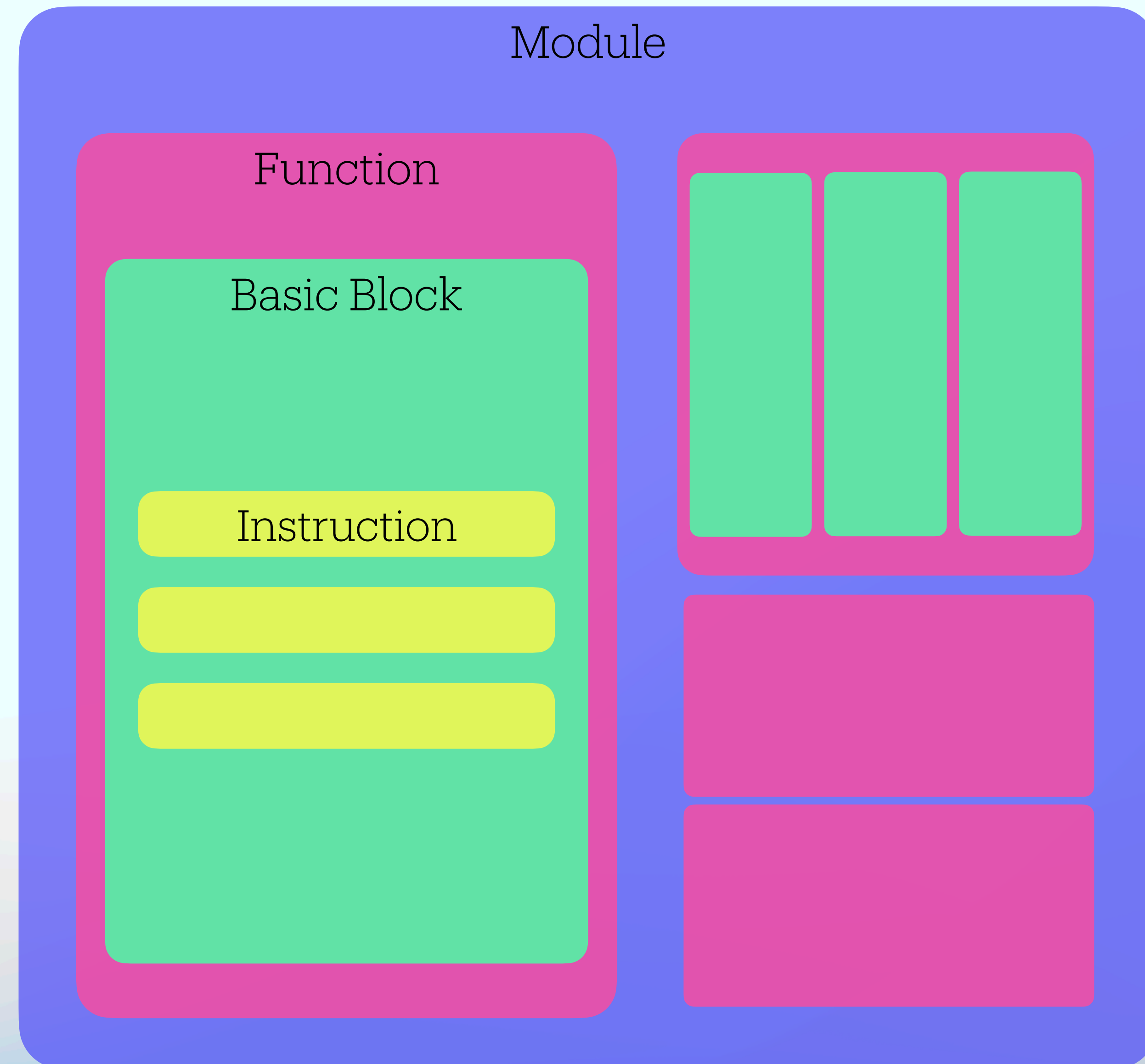
- **Modules** are the top-level container of all LLVM (IR) objects. Contains a list of global variables, list of functions, symbol table, etc.
- A **Function** is a procedure consisting of multiple basic blocks, usually connected via logical branches and jumps.
- A **Basic Block** is a continuous code block without any branch or jump.



M -> F -> BB -> I

Building blocks of LLVM IR

- **Modules** are the top-level container of all LLVM (IR) objects. Contains a list of global variables, list of functions, symbol table, etc.
- A **Function** is a procedure consisting of multiple basic blocks, usually connected via logical branches and jumps.
- A **Basic Block** is a continuous code block without any branch or jump.
- An **Instruction** is a single operation that a processor can execute per cycle.*



LLVM C++ API

- LLVM provides a C++ API that can be used to parse and modify the LLVM bitcode.
- You will notice the following program pattern a lot in LLVM Passes -

```
for (auto &F : M) { // Iterate through each Function in the Module.  
    for (auto &BB : F) { // Iterate through each BasicBlock in the Function.  
        for (auto &I : BB) { // Iterate through each Instruction in the BasicBlock.  
  
            errs() << I.getNumOperands(); // Get number of operands (uint) of the Instruction.  
  
        }  
    }  
}
```

LLVM Pass : Static Call Counter Pass

Writing a LLVM analysis pass.

```
#define AES_TE4_3(x) AES_PRECOMP_TE4_3[(x)]

#define CryptoUtils_POOL_SIZE (0x1 << 17) // 2^17

// SHA256
/* Various logical functions */
#define Ch(x, y, z) (z ^ (x & (y ^ z)))
#define Maj(x, y, z) (((x | y) & z) | (x & y))
#define __S(x, n) RORc((x), (n))
#define R1(x, n) (((x) & 0xFFFFFFFFUL) >> (n))
#define Sigma0(x) (__S(x, 2) ^ __S(x, 13) ^ __S(x, 22))
#define Sigma1(x) (__S(x, 6) ^ __S(x, 11) ^ __S(x, 25))
#define Gamma0(x) (__S(x, 7) ^ __S(x, 18) ^ R1(x, 3))
#define Gamma1(x) (__S(x, 17) ^ __S(x, 19) ^ R1(x, 10))
#define MIN(x, y) (((x) < (y)) ? (x) : (y))

#define RND(a, b, c, d, e, f, g, h, i, ki) \
    t0 = h + Sigma1(e) + Ch(e, f, g) + ki + W[i]; \
    t1 = Sigma0(a) + Maj(a, b, c); \
    d += t0; \
    h = t0 + t1;

#define RORc(x, y) \
    (((((unsigned long) (x) & 0xFFFFFFFFUL) >> (unsigned long) ((y) & 31)) \
    ((unsigned long) (x) << (unsigned long) (32 - ((y) & 31)))) & \
    0xFFFFFFFFUL)

class CryptoUtils {
public:
```

Functions and calls in LLVM IR



Functions and calls in LLVM IR

```
; Function Attrs: mustprogress noline nounwind optnone ssp uwtable(sync)
define noundef i32 @_Z3Addi(i32 noundef %0, i32 noundef %1) #0 {
    %3 = alloca i32, align 4
    %4 = alloca i32, align 4
    store i32 %0, ptr %3, align 4
    store i32 %1, ptr %4, align 4
    %5 = load i32, ptr %3, align 4
    %6 = load i32, ptr %4, align 4
    %7 = add nsw i32 %5, %6
    ret i32 %7
}
```

Functions and calls in LLVM IR

```
; Function Attrs: mustprogress noline nounwind optnone ssp uwtable(sync)
define noundef i32 @_Z3Addii(i32 noundef %0, i32 noundef %1) #0 {
    %3 = alloca i32, align 4
    %4 = alloca i32, align 4
    store i32 %0, ptr %3, align 4
    store i32 %1, ptr %4, align 4
    %5 = load i32, ptr %3, align 4
    %6 = load i32, ptr %4, align 4
    %7 = add nsw i32 %5, %6
    ret i32 %7
}
```

```
%22 = load i32, ptr %6, align 4
%23 = load i32, ptr %7, align 4
%24 = call noundef i32 @_Z3Addii(i32 noundef %22, i32 noundef %23)
```

Creating a logic for our pass

StaticFunctionCallCounter

Creating a logic for our pass

StaticFunctionCallCounter

1. Create a structure to hold results:

Creating a logic for our pass

StaticFunctionCallCounter

1. Create a structure to hold results:

```
llvm::MapVector<const llvm::Function *, unsigned int>
```

Creating a logic for our pass

StaticFunctionCallCounter

1. Create a structure to hold results:

```
llvm::MapVector<const llvm::Function *, unsigned int>
```

2. Iterate all the instructions in the LLVM IR. (Module->Function->BasicBlock->Inst.)

Creating a logic for our pass

StaticFunctionCallCounter

1. Create a structure to hold results:

```
llvm::MapVector<const llvm::Function *, unsigned int>
```

2. Iterate all the instructions in the LLVM IR. (Module->Function->BasicBlock->Inst.)

3. Look for Call Instructions. (Is a direct call?)

Creating a logic for our pass

StaticFunctionCallCounter

1. Create a structure to hold results:

```
llvm::MapVector<const llvm::Function *, unsigned int>
```

2. Iterate all the instructions in the LLVM IR. (Module->Function->BasicBlock->Inst.)

3. Look for Call Instructions. (Is a direct call?)

4. Get the function object.

Creating a logic for our pass

StaticFunctionCallCounter

1. Create a structure to hold results:

```
llvm::MapVector<const llvm::Function *, unsigned int>
```

2. Iterate all the instructions in the LLVM IR. (Module->Function->BasicBlock->Inst.)

3. Look for Call Instructions. (Is a direct call?)

4. Get the function object.

5. If the function is not in the MapVector, add it to the vector and initialize with 0.

Creating a logic for our pass

StaticFunctionCallCounter

1. Create a structure to hold results:

```
llvm::MapVector<const llvm::Function *, unsigned int>
```

2. Iterate all the instructions in the LLVM IR. (Module->Function->BasicBlock->Inst.)

3. Look for Call Instructions. (Is a direct call?)

4. Get the function object.

5. If the function is not in the MapVector, add it to the vector and initialize with 0.

6. If it is there, increment the call counter by one.

Creating a logic for our pass

StaticFunctionCallCounter

1. Create a structure to hold results:

```
llvm::MapVector<const llvm::Function *, unsigned int>
```

2. Iterate all the instructions in the LLVM IR. (Module->Function->BasicBlock->Inst.)

3. Look for Call Instructions. (Is a direct call?)

4. Get the function object.

5. If the function is not in the MapVector, add it to the vector and initialize with 0.

6. If it is there, increment the call counter by one.

7. Get analysis results.

Creating a logic for our pass

StaticFunctionCallCounter

1. Create a structure to hold results:

```
llvm::MapVector<const llvm::Function *, unsigned int>
```

2. Iterate all the instructions in the LLVM IR. (Module->Function->BasicBlock->Inst.)

3. Look for Call Instructions. (Is a direct call?)

4. Get the function object.

5. If the function is not in the MapVector, add it to the vector and initialize with 0.

6. If it is there, increment the call counter by one.

7. Get analysis results.

8. Store the result in a CSV file - {Name, CallCount}

Code Walk

LLVM Obfuscation Edu. -
GitHub

```
[15]: 0x00010000UL, [16]: 0x00008000UL, [17]: 0x00004000UL, [18]: 0x00002000UL,
[20]: 0x00000800UL, [21]: 0x00000400UL, [22]: 0x00000200UL, [23]: 0x00000100UL,
[25]: 0x00000040UL, [26]: 0x00000020UL, [27]: 0x00000010UL, [28]: 0x00000008UL,
[30]: 0x00000002UL, [31]: 0x00000001UL};
```

```
CryptoUtils::CryptoUtils() { seeded = false; }
```

```
unsigned CryptoUtils::scramble32(const unsigned in, const char key[16]) {
    assert(key != NULL && "CryptoUtils::scramble key=NULL");
```

```
    unsigned tmpA, tmpB;
```

```
    // Orr, Nathan or Adi can probably break it, but who cares?
```

```
    // Round 1
```

```
    tmpA = 0x0;
```

```
    tmpA ^= AES_PRECOMP_TE0[((in >> 24) ^ key[0]) & 0xFF];
```

```
    tmpA ^= AES_PRECOMP_TE1[((in >> 16) ^ key[1]) & 0xFF];
```

```
    tmpA ^= AES_PRECOMP_TE2[((in >> 8) ^ key[2]) & 0xFF];
```

```
    tmpA ^= AES_PRECOMP_TE3[((in >> 0) ^ key[3]) & 0xFF];
```

```
    // Round 2
```

```
    tmpB = 0x0;
```

```
    tmpB ^= AES_PRECOMP_TE0[((tmpA >> 24) ^ key[4]) & 0xFF];
```

```
    tmpB ^= AES_PRECOMP_TE1[((tmpA >> 16) ^ key[5]) & 0xFF];
```

```
    tmpB ^= AES_PRECOMP_TE2[((tmpA >> 8) ^ key[6]) & 0xFF];
```

```
    tmpB ^= AES_PRECOMP_TE3[((tmpA >> 0) ^ key[7]) & 0xFF];
```

```
    // Round 3
```

```
    tmpA = 0x0;
```

```
    tmpA ^= AES_PRECOMP_TE0[((tmpB >> 24) ^ key[8]) & 0xFF];
```

```
    tmpA ^= AES_PRECOMP_TE1[((tmpB >> 16) ^ key[9]) & 0xFF];
```

```
    tmpA ^= AES_PRECOMP_TE2[((tmpB >> 8) ^ key[10]) & 0xFF];
```

```
    tmpA ^= AES_PRECOMP_TE3[((tmpB >> 0) ^ key[11]) & 0xFF];
```

```
    // Round 4
```

Class Definition

```
StaticFunctionCallAnalysisPass.h

StaticFunctionCallAnalysisPass.h x
26  #ifndef LLVMOBFUSCATIONPASS_STATICFUNCTIONCALLANALYSISPASS_H
27  #define LLVMOBFUSCATIONPASS_STATICFUNCTIONCALLANALYSISPASS_H
28
29  > #include ...
36
37  class StaticFunctionCallAnalysisPass : public llvm::AnalysisInfoMixin<StaticFunctionCallAnalysisPass> {
38  public:
39      // Type def. for analysis pass manager
40      // Result will be in a MapVector where each Vector element is map of NameOfTheFunction->NumberOfCalls
41      using Result = llvm::MapVector<const llvm::Function *, unsigned>;
42      llvm::MapVector<const llvm::Function *, unsigned> run(llvm::Module &M, llvm::ModuleAnalysisManager &);
43      llvm::MapVector<const llvm::Function *, unsigned> runOnModule(llvm::Module &M);
44
45      static bool isRequired() { return true; }
46
47  private:
48      // For internal LLVM analysis identification
49      static llvm::AnalysisKey Key;
50      friend struct llvm::AnalysisInfoMixin<StaticFunctionCallAnalysisPass>;
51  };
52
53  #endif//LLVMOBFUSCATIONPASS_STATICFUNCTIONCALLANALYSISPASS_H

<no default server> 53:61 LF UTF-8 .clang-tidy 4 spaces C++: StaticFunctionCallAnalysisPass | Debug
```

runOnModule(llvm::Module &M)

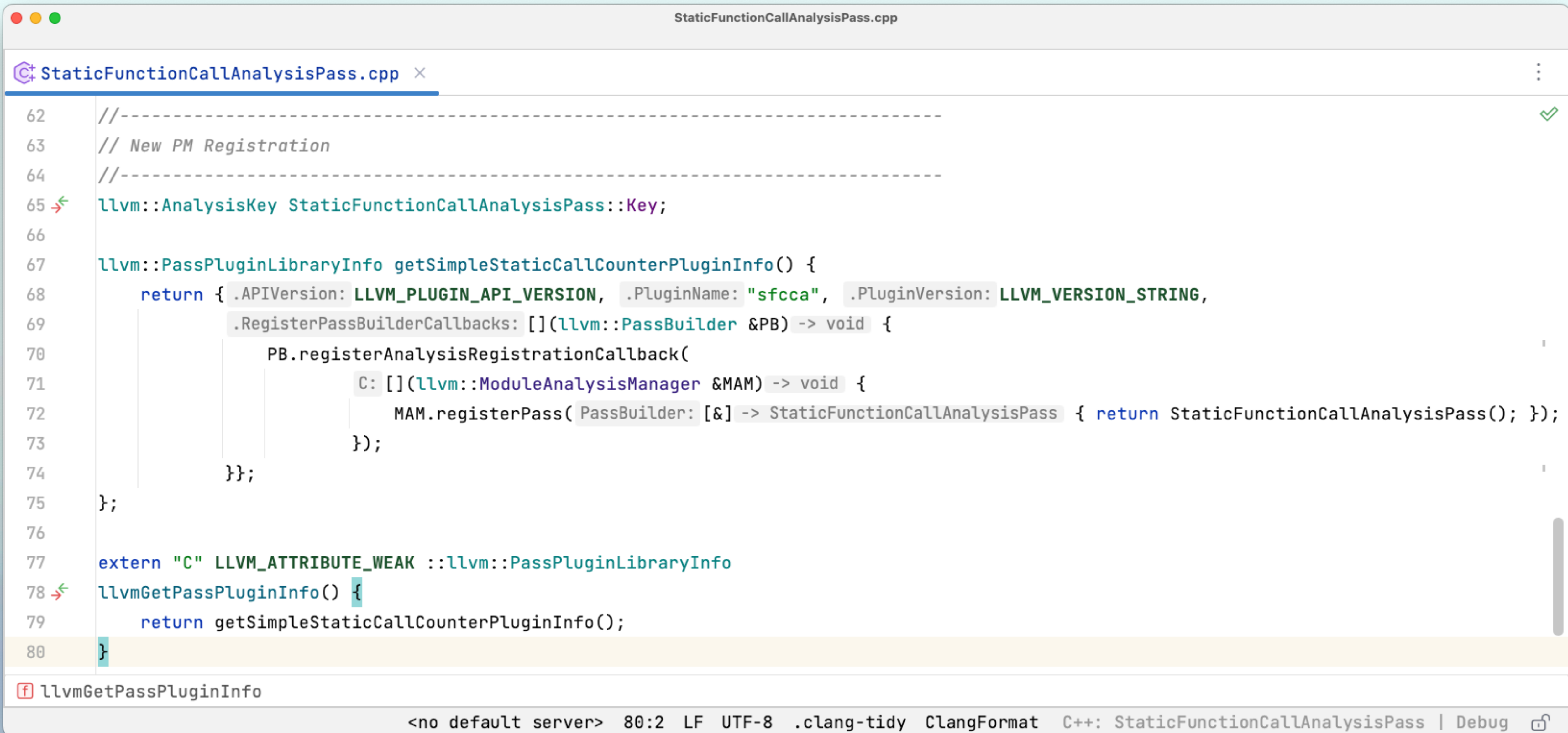
```
StaticFunctionCallAnalysisPass.cpp

28 using SCCReturnType = llvm::MapVector<const llvm::Function *, unsigned>;
29 //-----
30 // StaticCallCounter Implementation
31 //-----
32 ← SCCReturnType StaticFunctionCallAnalysisPass::runOnModule(llvm::Module &M) {
33     SCCReturnType ResultMapVector;
34     for (auto &TargetFunction: M) {
35         for (auto &TargetBasicBlock: TargetFunction) {
36             for (auto &TargetInstruction: TargetBasicBlock) {
37
38                 // dynamic_cast will fail if the instruction is not a Call.
39                 if (auto *CallBasePointer: CallBase * = llvm::dyn_cast<llvm::CallBase>(&TargetInstruction)) {
40
41                     // If callBasePointer is a direct function call then CalleeFunctionPointer will be not null.
42                     if (auto *CalleeFunctionPointer: Function * = CallBasePointer->getCalledFunction()) {
43                         // We have a direct function call - update the count for the fun. being called.
44                         auto *StaticCallCount = ResultMapVector.find(Key: CalleeFunctionPointer);
45                         if (ResultMapVector.end() == StaticCallCount) {
46                             StaticCallCount = ResultMapVector.insert(KV: std::make_pair(& CalleeFunctionPointer, t2: 0)).first;
47                         }
48                         ++StaticCallCount->second;
49                     }
50                 }
51             }
52         }
53     }
54     return ResultMapVector;
55 }

StaticFunctionCallAnalysisPass::runOnModule

<no default server> 54:2 LF UTF-8 .clang-tidy 4 spaces C++: StaticFunctionCallAnalysisPass | Debug
```

Pass Registration



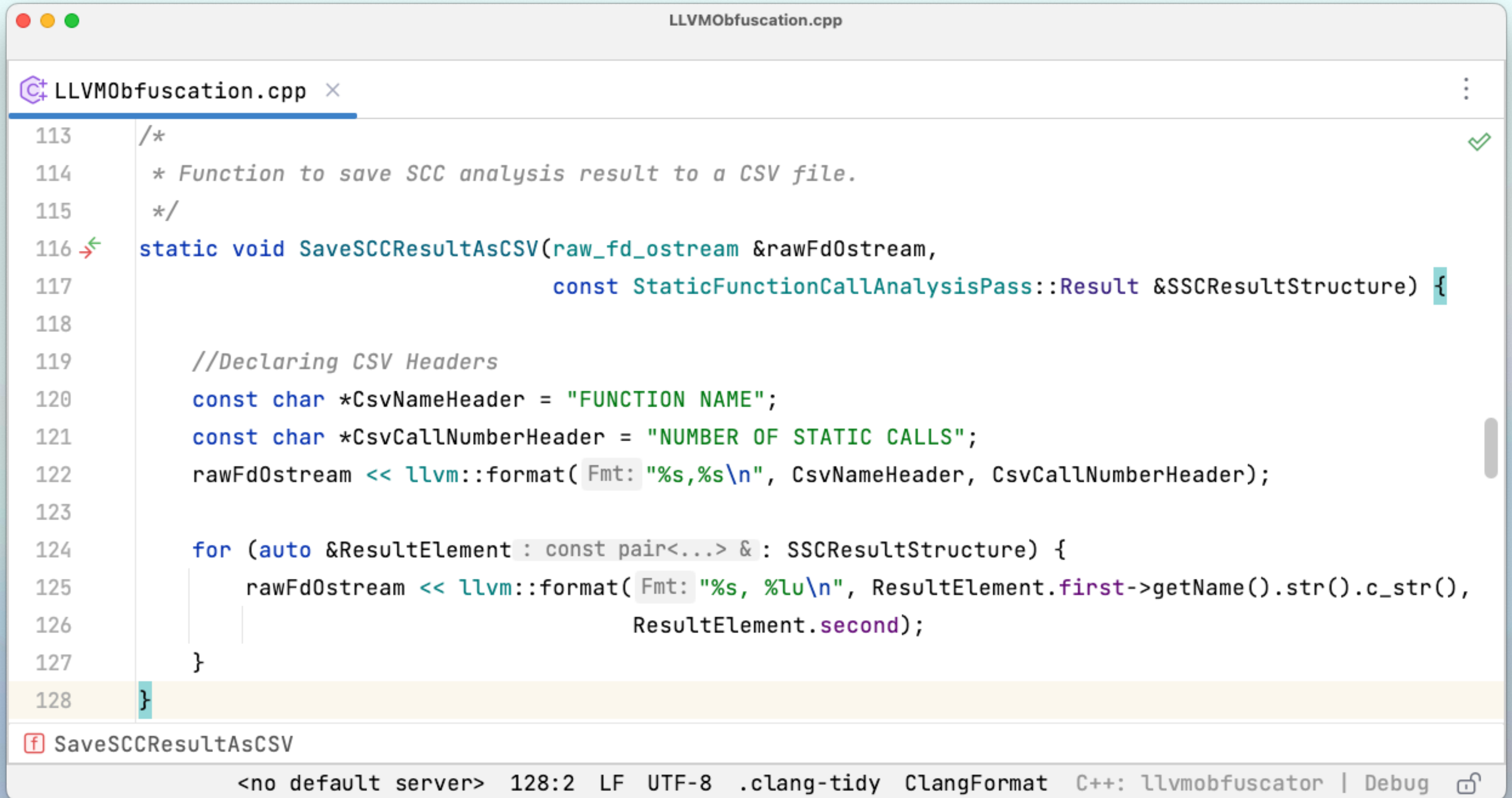
The screenshot shows a code editor window titled "StaticFunctionCallAnalysisPass.cpp". The code defines a new LLVM pass. Lines 62-64 are comments indicating a new pass registration. Line 65 declares the analysis key. Lines 67-75 define the plugin information and registration callbacks. Line 77 declares the external LLVM API function. Lines 78-80 implement the function to return the plugin information.

```
62 //-----
63 // New PM Registration
64 //-----
65 llvm::AnalysisKey StaticFunctionCallAnalysisPass::Key;
66
67 llvm::PassPluginLibraryInfo getSimpleStaticCallCounterPluginInfo() {
68     return { .APIVersion: LLVM_PLUGIN_API_VERSION, .PluginName: "sfcca", .PluginVersion: LLVM_VERSION_STRING,
69             .RegisterPassBuilderCallbacks: [](llvm::PassBuilder &PB) -> void {
70                 PB.registerAnalysisRegistrationCallback(
71                     C: [](llvm::ModuleAnalysisManager &MAM) -> void {
72                         MAM.registerPass( PassBuilder: [&] -> StaticFunctionCallAnalysisPass { return StaticFunctionCallAnalysisPass(); });
73                     });
74             });
75 };
76
77 extern "C" LLVM_ATTRIBUTE_WEAK ::llvm::PassPluginLibraryInfo
78 llvmGetPassPluginInfo() {
79     return getSimpleStaticCallCounterPluginInfo();
80 }
```

llvmGetPassPluginInfo

<no default server> 80:2 LF UTF-8 .clang-tidy ClangFormat C++: StaticFunctionCallAnalysisPass | Debug

Save SCC Result as CSV



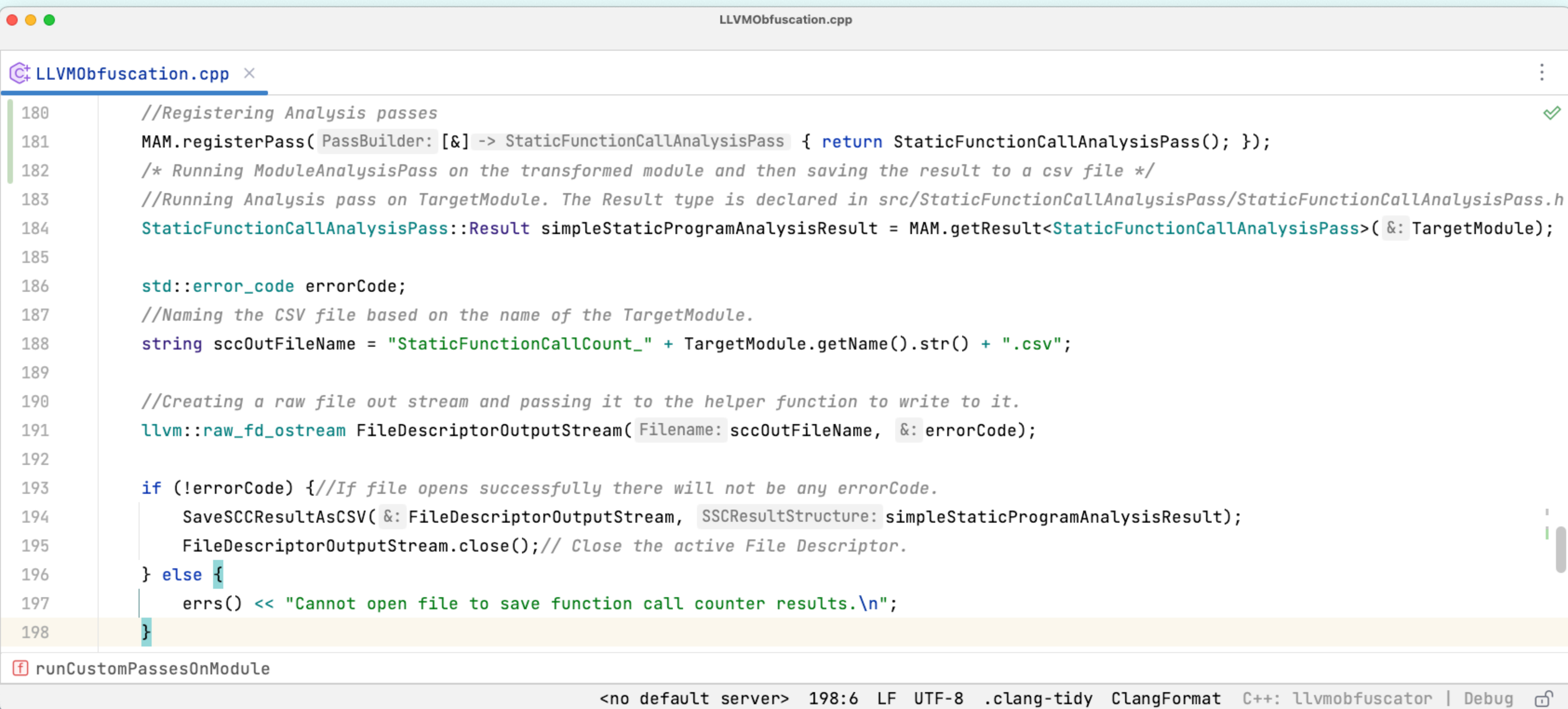
The screenshot shows a code editor window titled "LLVMObfuscation.cpp". The code is in C++ and implements a function to save the results of a Static Single Call (SCC) analysis as a CSV file. The function is named "SaveSCCResultAsCSV" and takes two parameters: "raw_fd_ostream &rawFdOstream" and "const StaticFunctionCallAnalysisPass::Result &SSCResultStructure". The function declares two CSV headers: "FUNCTION NAME" and "NUMBER OF STATIC CALLS". It then iterates over the "SSCResultStructure" and writes each result element to the CSV file in the format "FUNCTION NAME, NUMBER OF STATIC CALLS".

```
113  /*
114  * Function to save SCC analysis result to a CSV file.
115  */
116  static void SaveSCCResultAsCSV(raw_fd_ostream &rawFdOstream,
117                                const StaticFunctionCallAnalysisPass::Result &SSCResultStructure) {
118
119      //Declaring CSV Headers
120      const char *CsvNameHeader = "FUNCTION NAME";
121      const char *CsvCallNumberHeader = "NUMBER OF STATIC CALLS";
122      rawFdOstream << llvm::format(Fmt: "%s,%s\n", CsvNameHeader, CsvCallNumberHeader);
123
124      for (auto &ResultElement : const pair<...> & : SSCResultStructure) {
125          rawFdOstream << llvm::format(Fmt: "%s, %lu\n", ResultElement.first->getName().str().c_str(),
126                                     ResultElement.second);
127      }
128  }
```

SaveSCCResultAsCSV

<no default server> 128:2 LF UTF-8 .clang-tidy ClangFormat C++: llvmbfuscator | Debug

Registering and running the analysis pass

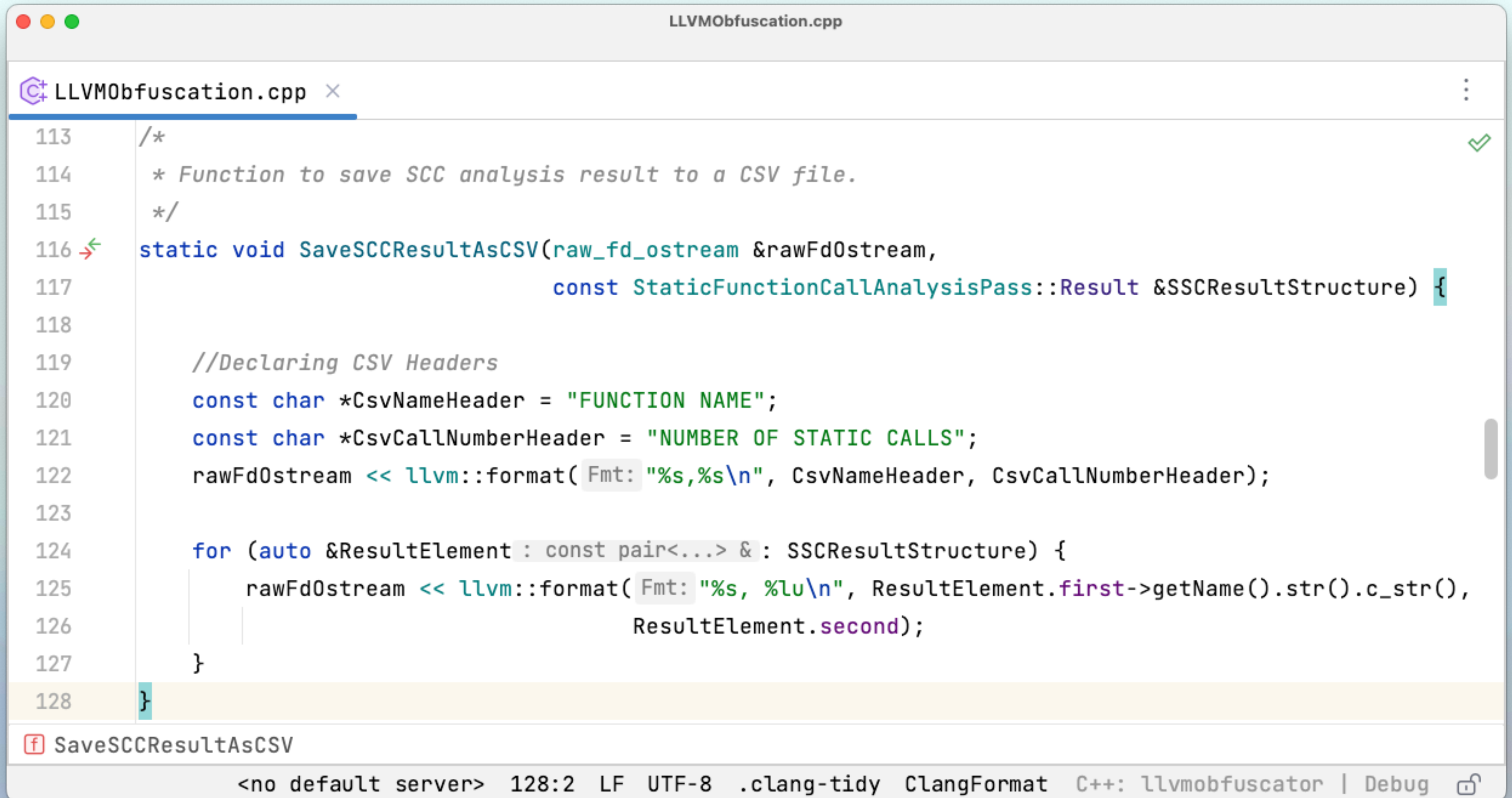


```
LLVMObfuscation.cpp x
180 //Registering Analysis passes
181 MAM.registerPass( PassBuilder: [&] -> StaticFunctionCallAnalysisPass { return StaticFunctionCallAnalysisPass(); });
182 /* Running ModuleAnalysisPass on the transformed module and then saving the result to a csv file */
183 //Running Analysis pass on TargetModule. The Result type is declared in src/StaticFunctionCallAnalysisPass/StaticFunctionCallAnalysisPass.h
184 StaticFunctionCallAnalysisPass::Result simpleStaticProgramAnalysisResult = MAM.getResult<StaticFunctionCallAnalysisPass>(& TargetModule);
185
186 std::error_code errorCode;
187 //Naming the CSV file based on the name of the TargetModule.
188 string sccOutFileName = "StaticFunctionCallCount_" + TargetModule.getName().str() + ".csv";
189
190 //Creating a raw file out stream and passing it to the helper function to write to it.
191 llvm::raw_fd_ostream FileDescriptorOutputStream( Filename: sccOutFileName, & errorCode);
192
193 if (!errorCode) { //If file opens successfully there will not be any errorCode.
194     SaveSCCResultAsCSV( & FileDescriptorOutputStream, SSCResultStructure: simpleStaticProgramAnalysisResult);
195     FileDescriptorOutputStream.close(); // Close the active File Descriptor.
196 } else {
197     errs() << "Cannot open file to save function call counter results.\n";
198 }
```

runCustomPassesOnModule

<no default server> 198:6 LF UTF-8 .clang-tidy ClangFormat C++: llvmbfuscator | Debug

Save SCC Result as CSV



```
LLVMObfuscation.cpp

113  /*
114  * Function to save SCC analysis result to a CSV file.
115  */
116  static void SaveSCCResultAsCSV(raw_fd_ostream &rawFdOstream,
117                                const StaticFunctionCallAnalysisPass::Result &SSCResultStructure) {
118
119      //Declaring CSV Headers
120      const char *CsvNameHeader = "FUNCTION NAME";
121      const char *CsvCallNumberHeader = "NUMBER OF STATIC CALLS";
122      rawFdOstream << llvm::format(Fmt: "%s,%s\n", CsvNameHeader, CsvCallNumberHeader);
123
124      for (auto &ResultElement : const pair<...> & : SSCResultStructure) {
125          rawFdOstream << llvm::format(Fmt: "%s, %lu\n", ResultElement.first->getName().str().c_str(),
126                                     ResultElement.second);
127      }
128  }
```

SaveSCCResultAsCSV

<no default server> 128:2 LF UTF-8 .clang-tidy ClangFormat C++: llvmbfuscator | Debug

Demonstration

```
#define AES_TE4_3(x) AES_PRECOMP_TE4_3[(x)]

#define CryptoUtils_POOL_SIZE (0x1 << 17) // 2^17

// SHA256
/* Various logical functions */
#define Ch(x, y, z) (z ^ (x & (y ^ z)))
#define Maj(x, y, z) (((x | y) & z) | (x & y))
#define __S(x, n) RORc((x), (n))
#define R1(x, n) (((x) & 0xFFFFFFFFUL) >> (n))
#define Sigma0(x) (__S(x, 2) ^ __S(x, 13) ^ __S(x, 22))
#define Sigma1(x) (__S(x, 6) ^ __S(x, 11) ^ __S(x, 25))
#define Gamma0(x) (__S(x, 7) ^ __S(x, 18) ^ R1(x, 3))
#define Gamma1(x) (__S(x, 17) ^ __S(x, 19) ^ R1(x, 10))
#define MIN(x, y) (((x) < (y)) ? (x) : (y))

#define RND(a, b, c, d, e, f, g, h, i, ki) \
    t0 = h + Sigma1(e) + Ch(e, f, g) + ki + W[i]; \
    t1 = Sigma0(a) + Maj(a, b, c); \
    d += t0; \
    h = t0 + t1;

#define RORc(x, y) \
    (((((unsigned long) (x) & 0xFFFFFFFFUL) >> (unsigned long) ((y) & 31)) \
    ((unsigned long) (x) << (unsigned long) (32 - ((y) & 31)))) & \
    0xFFFFFFFFUL)

class CryptoUtils {
public:
```

Resources

Codebase, Practice Ex., Slides.

```
#define AES_TE4_3(x) AES_PRECOMP_TE4_3[(x)]

#define CryptoUtils_POOL_SIZE (0x1 << 17) // 2^17

// SHA256
/* Various logical functions */
#define Ch(x, y, z) (z ^ (x & (y ^ z)))
#define Maj(x, y, z) (((x | y) & z) | (x & y))
#define __S(x, n) RORc((x), (n))
#define R1(x, n) (((x) & 0xFFFFFFFFUL) >> (n))
#define Sigma0(x) (__S(x, 2) ^ __S(x, 13) ^ __S(x, 22))
#define Sigma1(x) (__S(x, 6) ^ __S(x, 11) ^ __S(x, 25))
#define Gamma0(x) (__S(x, 7) ^ __S(x, 18) ^ R1(x, 3))
#define Gamma1(x) (__S(x, 17) ^ __S(x, 19) ^ R1(x, 10))
#define MIN(x, y) (((x) < (y)) ? (x) : (y))

#define RND(a, b, c, d, e, f, g, h, i, ki) \
    t0 = h + Sigma1(e) + Ch(e, f, g) + ki + W[i]; \
    t1 = Sigma0(a) + Maj(a, b, c); \
    d += t0; \
    h = t0 + t1;

#define RORc(x, y) \
    (((((unsigned long) (x) & 0xFFFFFFFFUL) >> (unsigned long) ((y) & 31)) \
    ((unsigned long) (x) << (unsigned long) (32 - ((y) & 31)))) & \
    0xFFFFFFFFUL)

class CryptoUtils {
public:
```

LLVM Obfuscation Edu. : GitHub

- Intentionally simple and minimal codebase.
- Play around with analysis results, or add a new obfuscation routine.



References

- a) "What is a compiler" - CS4620, Prof. Matthew Dwyer; University of Virginia.
- b) Lattner, Chris, and Vikram Adve. "LLVM: A compilation framework for lifelong program analysis & transformation." International symposium on code generation and optimization, 2004. CGO 2004.. IEEE, 2004.
- c) Junod, Pascal, et al. "Obfuscator-LLVM--software protection for the masses." 2015 IEEE/ACM 1st international workshop on software protection. IEEE, 2015.